

TEMA 2. Encapsulación

1. Encapsulación y ocultación de información en POO

En **Programación Orientada a Objetos**, la **encapsulación** y la **ocultación de información** buscan:

☒ Encapsulación

Agrupar datos (*atributos*) y comportamientos (*métodos*) dentro de una misma unidad llamada **clase**, de manera que el objeto controle cómo se accede y modifica su estado interno.

☒ Ocultación de información (information hiding)

Restringir el acceso directo a los datos internos del objeto, exponiendo solo lo necesario a través de métodos públicos (getters, setters, operaciones).

Esto permite **proteger el estado interno** del objeto frente a usos incorrectos o no deseados.

Ventajas de la ocultación de información

1. Mayor seguridad y control

Evita que otras partes del programa modifiquen los atributos de forma incorrecta.

2. Permite mantener invariantes internas

El objeto puede validar los datos antes de aceptarlos.

3. Reduce el acoplamiento

El exterior solo conoce la interfaz pública, no los detalles internos.

4. Facilita la modificación del código

Los atributos internos pueden cambiar sin afectar a quienes usan la clase.

5. Mejor mantenimiento y legibilidad

La interfaz pública suele ser más simple y clara que la implementación interna.

2. Interfaz pública de un objeto o clase en POO

La **interfaz pública** es el **conjunto de métodos y atributos accesibles desde fuera de la clase**.

Es decir, es *lo que un objeto ofrece al exterior* para que otros objetos o partes del programa interactúen con él.

Normalmente incluye:

- Métodos públicos (**public**)
- Atributos públicos (aunque en buen diseño suelen evitarse)
- Constantes públicas
- Cualquier elemento visible desde fuera de la clase

La interfaz pública define **qué puede hacer** un objeto, no **cómo lo hace** internamente.

Relación con la ocultación de información

La **ocultación de información** consiste en mantener los detalles internos de la clase (atributos, lógica interna) **ocultos**, normalmente usando modificadores como **private** o **protected**.

Esto conecta directamente con la interfaz pública porque:

- La **interfaz pública** expone **solo lo necesario** para usar el objeto.
- La **ocultación** protege la implementación interna para que no pueda ser manipulada directamente.
- Así, otros objetos solo pueden interactuar a través de la **interfaz pública**, no con los detalles internos.

En otras palabras:

La interfaz pública es la puerta controlada de acceso;
la ocultación de información es la pared que protege todo lo que hay detrás.

3. ¿Por qué hay que diseñar con cuidado la interfaz pública de una clase?

Porque la **interfaz pública es el contrato** entre la clase y el resto del programa.

Todo el código externo que utilice la clase **depende** de ese contrato.

Por eso debe diseñarse con cuidado:

- Define cómo se usa la clase.
 - Marca qué operaciones se permiten y cuáles no.
 - Afecta directamente a la simplicidad, coherencia y seguridad del código.
 - Un buen diseño reduce errores y hace la clase fácil de entender y de mantener.
-

¿Es fácil cambiarla?

Generalmente, **no**, no es fácil cambiarla.

Si modificas la interfaz pública (por ejemplo, cambiando nombres de métodos, tipos de parámetros o eliminando funciones), todo el código que la use puede **dejar de compilar o comportarse de forma incorrecta**.

Por eso se considera una decisión importante:

Una vez publicada, la interfaz pública debe permanecer estable.

4. ¿Qué son las invariantes de clase?

Las **invariantes de clase** son **condiciones lógicas que siempre deben cumplirse** para que los objetos de una clase estén en un **estado válido**.

- Se cumplen **después de construir el objeto**.
- Se cumplen **antes y después de ejecutar cualquier método público**.
- Se rompen temporalmente *dentro* de métodos privados, pero deben restablecerse antes de volver a la interfaz pública.

Ejemplo típico:

En una clase `CuentaBancaria`, una invariante sería:

- *el saldo nunca puede ser negativo.*

¿Por qué la ocultación de información ayuda a mantener las invariantes?

La **ocultación de información** impide que el exterior modifique directamente los atributos internos. Esto ayuda porque:

1. Controlas todos los cambios del estado desde dentro de la clase

Si los atributos son privados, solo se pueden modificar mediante métodos controlados (setters, operaciones), donde puedes verificar que la invariante se cumpla.

2. Evitas modificaciones incorrectas desde el exterior

Si el atributo fuera público, cualquier parte del código podría cambiarlo a un valor que rompa la invariante, por ejemplo:

```
cuenta.saldo = -500;    // Rompe la invariante
```

3. Garantizas que el objeto nunca queda en un estado inconsistente

Como todos los cambios pasan por la lógica interna de la clase, puedes:

- validar datos,
- prevenir valores inválidos,
- reconstruir la invariante si algo falla.

En resumen

Las invariantes definen lo que “siempre debe ser cierto” en un objeto.

La ocultación de información evita que el exterior pueda violarlas.

5. Ejemplo de una clase **Punto** en **Java**, con dos coordenadas, **x** e **y**, de tipo **double**, con un método **calcularDistanciaAOrigen**, con uso de la ocultación de información.

Clase **Punto** (con ocultación de información)

```
public class Punto {
    // Atributos privados: ocultamos la información interna
    private double x;
    private double y;

    // Constructor: establece un estado válido inicial
    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
        // Aquí podríamos validar invariantes si hiciera falta
    }

    // Getters: exponen lectura controlada (parte de la interfaz pública)
    public double getX() {
        return x;
    }

    public double getY() {
        return y;
    }

    // Setters: exponen modificación controlada (podemos validar si queremos)
    public void setX(double x) {
        this.x = x;
    }

    public void setY(double y) {
        this.y = y;
    }

    // Método público: calcula la distancia al origen (0,0)
    public double calcularDistanciaAOrigen() {
        return Math.sqrt(x * x + y * y);
    }

    // (Opcional) Método de utilidad para depuración/impresión
    @Override
    public String toString() {
        return "Punto(" + x + ", " + y + ")";
    }
}
```

¿Cuál es la **interfaz pública** de la clase **Punto**?

La **interfaz pública** es todo aquello que otros pueden usar desde fuera de la clase.

En este caso, son los **miembros public**:

- `public Punto(double x, double y)` — constructor
- `public double getX()`
- `public double getY()`
- `public void setX(double x)`
- `public void setY(double y)`
- `public double calcularDistanciaAOrigen()`
- `public String toString()` (sobrescrito)

Los **atributos x e y** **NO** forman parte de la **interfaz pública** porque son **private**.

¿Qué significan **public** y **private**?

- **public**: El miembro (clase, método, constructor o atributo) es **accesible desde cualquier otra clase**. Es lo que compone la **interfaz pública**, el “contrato” de uso.
 - **private**: El miembro **solo es accesible desde dentro de la propia clase**. Sirve para **ocultar la información** y proteger el estado interno (nadie desde fuera puede tocar **x** o **y** directamente).
-

6. ☒ ¿A quiénes se pueden aplicar los modificadores **public** y **private** en Java?

En Java, los modificadores de acceso **public** y **private** pueden aplicarse a:

✓ 1. Atributos (variables de instancia o de clase)

```
private int edad;  
public String nombre;
```

✓ 2. Métodos

```
public void mover();  
private void calcular();
```

✓ 3. Constructores

```
public Punto(double x, double y) { ... }  
private Punto() { ... }
```

✓ 4. Clases internas (inner classes)

```
private class Nodo { ... }  
public class Gestor { ... }
```

! ¿Y a las clases normales (top-level)?

En clases “de nivel superior” (las que no están dentro de otra clase), **solo se puede usar:**

- `public`
- o ningún modificador (default / package-private)

No se puede usar `private` en una clase top-level.

Ejemplo válido:

```
public class MiClase { ... }  
class MiOtraClase { ... } // package-private
```

Ejemplo NO válido:

```
private class Error { ... } // ✗ No permitido
```

✓ Resumen rápido

Elemento Java	<code>public</code>	<code>private</code>
Atributos	✓	✓
Métodos	✓	✓
Constructores	✓	✓
Clases internas	✓	✓
Clases top-level	✓	✗

7. ☒ ¿Existen más tipos de visibilidad en POO? ¿Qué ocurre en Java? ¿Y en otros lenguajes?

En **POO**, la idea general es controlar quién puede acceder a los atributos y métodos de una clase. Aunque solemos hablar de **público** y **privado**, **muchos lenguajes ofrecen más niveles de visibilidad**.

Visibilidad en Java

Java define **4 niveles de visibilidad**:

✓ 1. **public**

Accesible desde *cualquier parte* del programa.

✓ 2. **private**

Accesible *solo* desde la propia clase.

✓ 3. **protected**

Accesible desde:

- la propia clase,
- las clases del mismo paquete,
- y las subclases (aunque estén en otro paquete).

✓ 4. **(default) o package-private**

Sin escribir ningún modificador.

Accesible únicamente desde el **mismo paquete**.

Ejemplo:

```
public class Ejemplo {  
    public int a;      // Público  
    private int b;     // Privado  
    protected int c;  // Protegido  
    int d;             // Default (package-private)  
}
```

¿Y en otros lenguajes?

Los lenguajes de POO suelen tener mecanismos parecidos, pero con diferencias importantes.

Python

Python *no implementa* visibilidad estricta.

Su filosofía es: "somos adultos responsables".

- Todo es público por defecto.
 - `__atributo` → se usa *name mangling* para indicar que es privado, pero no bloquea acceso real.
 - `_atributo` → convención: "esto es interno".
-

C++

Tiene **tres niveles**, similares a Java:

- `public`
- `private`
- `protected`

Pero con una diferencia importante:

el nivel se aplica a toda la clase por defecto hasta que se cambie, no en cada miembro.

También permite **clases amigas** (`friend`), que pueden acceder a sus privados.

Resumen final

- Sí, **existen más tipos de visibilidad** que público y privado en muchos lenguajes.
 - En **Java hay 4 niveles**: `public`, `private`, `protected` y `package-private`.
 - Otros lenguajes añaden otros niveles según su filosofía (como `internal`, `protected internal`, `friend`, etc.).
-
-

8. Los miembros de instancia privados de un objeto están ocultos para (a) otras clases o (b) otras instancias, aunque sean de la misma clase?

Respuesta breve:

En **Java**, los miembros de instancia `private` están **ocultos para otras clases (a)**, no para **otras instancias de la misma clase (b)**.

Desde **dentro de la propia clase**, un método puede **acceder a los campos privados de cualquier instancia de esa misma clase**, no solo a los suyos.

Ejemplo: `Punto` con `calcularDistanciaAPunto(Punto otro)`

```
public class Punto {  
    // Campos privados: ocultos para otras clases  
    private double x;
```



```

private double y;

public Punto(double x, double y) {
    this.x = x;
    this.y = y;
}

public double calcularDistanciaAOrigen() {
    return Math.sqrt(x * x + y * y);
}

// Nuevo método: calcula la distancia a otro Punto
public double calcularDistanciaAPunto(Punto otro) {
    // ✓ Desde dentro de la clase Punto, podemos acceder a 'otro.x' y
    // 'otro.y'
    // aunque sean 'private', porque 'otro' es también un Punto.
    double dx = this.x - otro.x; // acceso válido a campos privados de otra
    // instancia de la MISMA clase
    double dy = this.y - otro.y; // acceso válido
    return Math.sqrt(dx * dx + dy * dy);
}

@Override
public String toString() {
    return "Punto(" + x + ", " + y + ")";
}
}

```

¿Por qué esto es válido?

- La **visibilidad `private`** en Java **se evalúa por clase, no por instancia**. Es decir, **todo el código de la clase `Punto`** puede leer/escribir los campos `private` de **cualquier objeto `Punto`**, incluido el parámetro `otro`.
- Lo que **sí está prohibido** es que **otra clase distinta** (por ejemplo, `Main`, `Triangulo`, etc.) intente leer `p.x` o `p.y` directamente:

```

Punto p = new Punto(1, 2);
double xx = p.x; // ✗ Error de compilación: x tiene acceso privado en Punto

```

Conclusión

- **Correcta:** (a) **otras clases** no pueden acceder a los miembros `private`.
- **Incorrecta:** (b) **otras instancias de la misma clase** **sí** pueden ser accedidas desde métodos de la clase, aunque esos miembros sean `private`.

9. ¿Qué son los métodos *getter* y *setter* en los lenguajes orientados a objetos?

En POO, los métodos **getter** y **setter** son métodos usados para **acceder** y **modificar** atributos privados de una clase, respetando la *ocultación de información*.

◇ **Getter**

Un **getter** es un método que **devuelve** el valor de un atributo privado.
Su función es *leer* el valor.

Ejemplo en Java:

```
public double getX() {  
    return x;  
}
```

◇ **Setter**

Un **setter** es un método que **modifica** el valor de un atributo privado.
Su función es *escribir* o *actualizar* el valor, normalmente con validación.

Ejemplo:

```
public void setX(double x) {  
    this.x = x;  
}
```

¿Por qué existen?

Porque en un buen diseño de POO:

- los **atributos son privados** (**private**),
- y el acceso se controla **solo** a través de la **interfaz pública** de la clase.

Esto permite:

- ✓ Validar datos antes de asignarlos
 - ✓ Mantener invariantes
 - ✓ Evitar accesos inseguros
 - ✓ Poder cambiar la implementación sin romper el código externo
-
-

10. ¿A qué nos referimos con “seguridad” en POO?

Nos referimos a una **seguridad lógica**, también llamada:

- *seguridad de integridad*
- *seguridad del estado interno*
- *robustez del diseño*

Significa que:

- ✓ El objeto no puede quedar en un estado inconsistente

Porque nadie fuera de la clase puede modificar sus atributos libremente.

- ✓ Solo se permite modificar el estado a través de métodos controlados

Lo que permite validar datos, comprobar invariantes, impedir valores incorrectos, etc.

- ✓ Protege al programa contra errores del propio programador

No contra atacantes externos.

Lo que **NO** significa

La ocultación de información **no evita**:

- hacking
- ataques externos
- inyección de código
- exploits
- accesos ilegítimos al sistema

Esto pertenece al campo de la **ciberseguridad**, no a la POO.

Ejemplo sencillo

Si los atributos fueran públicos:

```
p.x = Double.NaN;    // El objeto puede quedar en un estado inválido
p.y = -9999999;      // Rompe invariantes
```

Con atributos privados:

```
p.setX(-5); // Aquí el setter podría impedir valores inválidos
```

El objeto **se protege de usos incorrectos**, no de ataques maliciosos.

★ Resumen final

La ocultación de información mejora la seguridad del *diseño* del programa, no la seguridad informática frente a hackers.

11. ☑ ¿Diferencia entre miembro de instancia y miembro de clase?

◇ Miembro de instancia

Es un **atributo o método que pertenece a cada objeto concreto**.

- Cada instancia tiene **su propia copia** del atributo.
- Solo existen cuando haces **new**.
- Se accede a ellos *a través de un objeto*.

Ejemplo:

```
private double x;    // miembro de instancia
private double y;    // miembro de instancia
```

Cada **Punto p1** y **Punto p2** tiene **sus propios x e y**.

◇ Miembro de clase (también llamado **estático**)

Es un atributo o método que **pertenece a la clase en sí, no a los objetos**.

- Hay **una única copia para toda la clase**, compartida por todas las instancias.
- Se declara con la palabra clave **static**.
- Se accede normalmente desde la clase, no desde un objeto.

Ejemplo:

```
private static int contador; // miembro de clase
```

Todos los objetos comparten **el mismo contador**.

☑ ¿Los miembros de clase también se pueden ocultar?

Sí.

Los miembros de clase (estáticos) **también pueden ser `private`**, igual que los de instancia.

```
public class Punto {  
    private static int contador = 0; // ocultado: solo accesible desde Punto  
  
    public static int getContador() {  
        return contador;  
    }  
}
```

✓ ¿Qué significa esto?

- Ninguna otra clase puede acceder directamente a `Punto.contador`.
- Solo la clase `Punto` puede leerlo o modificarlo.
- Desde fuera, solo pueden interactuar **a través del getter público**, si lo decides.

Es exactamente la **misma regla** que para los atributos no estáticos:

La visibilidad `private` restringe el acceso **a la clase**, no importa si es atributo de instancia o de clase.

★ Resumen rápido

Tipo de miembro	Pertenece a...	Número de copias	Palabra clave	¿Se puede ocultar con <code>private</code> ?
Miembro de instancia	cada objeto	una por objeto	(sin <code>static</code>)	✓ Sí
Miembro de clase	la clase	una para todos	<code>static</code>	✓ Sí

12. ☑ ¿Tiene sentido que los constructores sean privados?

Sí, **tiene sentido que los constructores sean privados**, aunque **no es lo más habitual**. Se usa en situaciones muy concretas y es una herramienta importante de diseño en POO.

Un constructor privado **impide crear objetos desde fuera de la clase**, lo cual puede ser útil en varios patrones de diseño.

◇ ¿Para qué sirve un constructor privado?

1. Para controlar completamente cuántas instancias se crean

Patrón típico: **Singleton**.

Solo se permite una única instancia.

```
public class Config {  
    private static Config instancia = new Config(); // única instancia  
  
    private Config() {} // constructor privado  
  
    public static Config getInstancia() {  
        return instancia;  
    }  
}
```

2. Para impedir que alguien cree instancias de una clase que no debe instanciarse

Ejemplo: clases con **métodos estáticos solamente** (utilidades).

```
public class Matematicas {  
    private Matematicas() {} // evita hacer new Matematicas()  
}
```

3. Para obligar a usar métodos estáticos "factory"

En lugar de **new**, la clase ofrece métodos como **crear(...)**, **desdeCadena(...)**, etc.

```
public class Punto {  
    private double x, y;  
  
    private Punto(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public static Punto crear(double x, double y) {  
        return new Punto(x, y);  
    }  
}
```

Esto da flexibilidad para:

- validar valores,
- devolver objetos cacheados,
- decidir qué tipo concreto crear.

★ Resumen final

Sí tiene sentido que los constructores sean privados, pero únicamente cuando quieres **controlar la creación de objetos**. Se usa en patrones como *Singleton*, *clases de utilidades* o *factory methods*.

13. ¿Cómo se indican los miembros de clase en Java?

Los **miembros de clase** (también llamados **estáticos**) se indican con la palabra clave **static**.

- **Pertenecen a la clase**, no a cada objeto.
- Hay **una única copia compartida** por todas las instancias.
- Se suelen **acceder con el nombre de la clase**: `Punto.getMaxX()`.

Ejemplo: **Punto** con máximos globales de **x** e **y**

- Guardamos en **miembros estáticos privados** los máximos de **x** e **y** vistos hasta ahora.
- Los actualizamos **en el constructor** y en los **setters** (si existen).
- Exponemos **getters estáticos públicos** para consultarlos.

```
public class Punto {  
    // Atributos de instancia (ocultación de información)  
    private double x;  
    private double y;  
  
    //  Miembros de clase (estáticos): comparten estado global de la clase  
    private static double maxX = Double.NEGATIVE_INFINITY;  
    private static double maxY = Double.NEGATIVE_INFINITY;  
  
    // Constructor  
    public Punto(double x, double y) {  
        this.x = x;  
        this.y = y;  
        actualizarMaximos(x, y);  
    }  
  
    // Getters de instancia  
    public double getX() { return x; }  
    public double getY() { return y; }
```

```
// Setters de instancia (si permites mutabilidad)
public void setX(double x) {
    this.x = x;
    actualizarMaximos(this.x, this.y); // por si cambia el máximo
}

public void setY(double y) {
    this.y = y;
    actualizarMaximos(this.x, this.y);
}

// Método público: distancia al origen
public double calcularDistanciaAOrigen() {
    return Math.sqrt(x * x + y * y);
}

// (Del punto 8) Distancia a otro punto
public double calcularDistanciaAPunto(Punto otro) {
    double dx = this.x - otro.x;
    double dy = this.y - otro.y;
    return Math.sqrt(dx * dx + dy * dy);
}

// ☒ Getters ESTÁTICOS (interfaz pública para los máximos globales)
public static double getMaxX() { return maxX; }
public static double getMaxY() { return maxY; }

// Método privado de clase para mantener la invariante de máximos
private static void actualizarMaximos(double x, double y) {
    if (x > maxX) maxX = x;
    if (y > maxY) maxY = y;
}

@Override
public String toString() {
    return "Punto(" + x + ", " + y + ")";
}
}
```

Cómo se usa

```
Punto p1 = new Punto(1.0, 5.0);
Punto p2 = new Punto(3.5, 2.0);
p1.setX(10.0);

System.out.println(Punto.getMaxX()); // 10.0
System.out.println(Punto.getMaxY()); // 5.0
```


14. ☒ Método factoría que redondea las coordenadas

```
public static Punto crearRedondeado(double x, double y) {  
    long rx = Math.round(x);  
    long ry = Math.round(y);  
    return new Punto(rx, ry);  
}
```

? ¿He usado `static`?

Sí.

Un **método factoría** debe ser `static` porque:

- pertenece a la **clase**, no a un objeto concreto,
- se usa como alternativa a `new`,
- se invoca así: `Punto.crearRedondeado(... , ...)`.

15. Cambia la implementación de `Punto`. En vez de dos `double`, emplea un array interno de dos posiciones, intentando no modificar la interfaz pública de la clase.

 **Ocultación de información:** el cambio es **totalmente interno**; el código cliente que usa la clase no necesita cambiar.

```
public class Punto {  
    // Representación interna: array de 2 posiciones [x, y]  
    private final double[] coords = new double[2];  
  
    // Constructor público (misma interfaz)  
    public Punto(double x, double y) {  
        coords[0] = x; // x  
        coords[1] = y; // y  
    }  
  
    // Getters públicos (misma interfaz)  
    public double getX() {  
        return coords[0];  
    }  
  
    public double getY() {  
        return coords[1];  
    }  
  
    // Setters públicos (misma interfaz)  
    public void setX(double x) {  
        coords[0] = x;  
    }  
}
```

```

    }

    public void setY(double y) {
        coords[1] = y;
    }

    // Método público: distancia al origen (misma interfaz)
    public double calcularDistanciaAOrigen() {
        double x = coords[0], y = coords[1];
        return Math.sqrt(x * x + y * y);
    }

    // Método público: distancia a otro punto (misma interfaz)
    public double calcularDistanciaAPunto(Punto otro) {
        double dx = this.coords[0] - otro.coords[0];
        double dy = this.coords[1] - otro.coords[1];
        return Math.sqrt(dx * dx + dy * dy);
    }

    @Override
    public String toString() {
        return "Punto(" + coords[0] + ", " + coords[1] + ")";
    }
}

```

☑ 16.1. Si un atributo tiene getter y setter públicos, ¿no es mejor declararlo público?

No.

Incluso si un atributo tiene getter y setter públicos, **no conviene declararlo público**.

¿Por qué?

Porque los getters y setters permiten:

✓ Validar valores

```

public void setEdad(int e) {
    if (e < 0) throw new IllegalArgumentException("Edad inválida");
    this.edad = e;
}

```

Si el atributo fuese `public`:

```

persona.edad = -100; // ✗ Imposible de evitar y rompe la clase

```

✓ Mantener invariantes de clase

La clase controla **cómo** se cambia su estado.

✓ Poder cambiar la implementación interna

Por ejemplo, pasar de `double x, y` a un array interno **sin romper el código cliente** (como hicimos en la pregunta 15).

✓ Encapsular la representación

La interfaz pública no revela detalles internos que podrían cambiar en el futuro.

☑ 16.2. ¿Cuál es la convención más habitual sobre los atributos?

La convención de diseño (muy fuerte en Java y la mayoría de lenguajes OO) es:

Los atributos deben ser *privados*.

private fields + public getters/setters (si se necesitan)

Solo se hacen públicos en casos excepcionales (por ejemplo, constantes públicas `public static final`).

☑ 16.3. ¿Tiene esto algo que ver con las invariantes de clase?

Sí, totalmente.

Las **invariantes de clase** son condiciones que siempre deben cumplirse para que el objeto esté en un estado válido.

Ejemplos:

- un saldo nunca puede ser negativo,
- un punto no puede tener coordenadas NaN,
- una fecha debe tener día válido, etc.

Si los atributos fueran públicos:

```
p.x = Double.NaN;           // ✗ Violación directa de invariante
p.y = -999999999;          // ✗ Nadie lo impide
```

Con **atributos privados + setters**, la clase puede protegerse:

```
public void setX(double x) {
    if (Double.isNaN(x)) throw new IllegalArgumentException();
    this.x = x;
}
```

La ocultación de información es necesaria para garantizar las invariantes.
Si los atributos fuesen públicos, las invariantes no se podrían asegurar.

★ Resumen final

- ✓ Aunque haya getters y setters públicos, los **atributos deben mantenerse privados**.
- ✓ Es la **convención estándar** en Java y en buena POO.
- ✓ Esto está directamente relacionado con las **invariantes de clase**, que solo la propia clase puede hacer cumplir.
- ✓ Atributos públicos debilitan la encapsulación, rompen la seguridad lógica y dificultan modificaciones futuras.

☑ 17.1. ¿Qué significa que una clase sea *immutable*?

Una clase es **immutable** cuando:

- Su **estado interno no puede cambiar después de crear el objeto**.
- No existen métodos que modifiquen los atributos.
- Todos sus atributos son normalmente **privados** y **final**.
- No tiene *setters*.

Ejemplo clásico: `String` en Java es **immutable**.

☑ 17.2. ¿Qué es un método modificador?

Un **método modificador** (*modifier method*) es cualquier método que **cambia el estado interno del objeto**.

Ejemplos típicos:

- Un `setter`
- Un método que aumente una coordenada:

```
public void mover(double dx, double dy) {  
    x += dx;  
    y += dy;  
}
```

! 17.3. ¿Un método modificador es siempre un setter?

No.

- Un **setter** es un tipo especial de método modificador.
- Pero **no todo método modificador es un setter**.

Ejemplo de modificador que NO es setter:

```
public void incrementarContador() {  
    contador++;  
}
```

☑ 17.4. ¿Tiene ventajas que una clase sea inmutable?

Sí, una clase inmutable tiene muchas ventajas:

✓ Mayor seguridad y simplicidad

Es imposible que cambie accidentalmente el estado.
Evita errores muy comunes.

✓ Hilos (multithreading) seguros

Las clases inmutables son **thread-safe** automáticamente.
No requieren sincronización.

✓ Más fáciles de razonar y depurar

Un objeto inmutable siempre representa los mismos datos.

✓ Pueden usarse como claves en mapas o elementos en sets

Como no cambian, su `hashCode` y su igualdad (`equals`) permanecen válidos.

✓ Facilitan cumplir invariantes

Si el estado no cambia, la invariante solo debe cumplirse en el constructor.

★ Resumen final

Concepto	Explicación
Clase inmutable	Una clase cuyo estado no cambia tras construirse.
Método modificador	Cualquier método que altere el estado del objeto.
¿Modificador = setter?	✗ No siempre. Todo setter es modificador, pero no todo modificador es setter.
Ventajas de inmutabilidad	Seguridad, simplicidad, seguridad en hilos, facilidad de uso y mantenimiento.

18. ¿Es recomendable incluir setters siempre?

No, **no es recomendable incluir métodos *setter* siempre ni como convención.**

De hecho, en un buen diseño orientado a objetos, **lo normal es no poner setters salvo que realmente hagan falta.**

Poner setters “por costumbre” es **mala práctica** y rompe varios principios de buen diseño en POO.

¿Por qué NO se deben incluir por defecto?

✓ 1. Rompen la encapsulación si permiten modificar todo sin control

Un setter indiscriminado deja la clase expuesta:

```
p.setX(-9999); // Puede romper invariantes
```

✓ 2. Numerosos objetos NO deberían poder cambiar después de construirse

Muchos objetos representan **valores**: puntos, fechas, dinero, colores...

En estos casos la inmutabilidad es más segura.

✓ 3. Dificultan mantener invariantes

Si los atributos pueden cambiar libremente, mantener la clase en un estado válido es más difícil.

✓ 4. Impiden optimizaciones internas

Si todo es modificable, la clase no puede:

- cachear valores,
- volverse inmutable,
- cambiar su representación interna (como en tu ejercicio 15),
- garantizar consistencia.

📦 Entonces, ¿cuál es la convención?

Convención estándar en Java y en POO:

Los atributos deben ser privados y solo se crean setters cuando realmente son necesarios.

No todos los atributos necesitan un setter.

Para muchos, lo correcto es **solo un getter**, o incluso **ninguno**.

🔗 ¿Tiene relación con las invariantes de clase?

Sí, totalmente.

Las invariantes son condiciones que deben cumplirse siempre para que el objeto esté en un estado válido.

- Si pones setters indiscriminados → **cualquier código externo puede romper la invariante.**
- Si limitas los setters o no los pones → la clase controla mejor que la invariante siempre se cumpla.

Cuantos menos setters, más fácil es asegurar las invariantes.

★ Resumen en una frase

✗ No, no es recomendable incluir setters siempre.

✓ Solo deben existir cuando son necesarios, porque pueden romper la encapsulación, la inmutabilidad y las invariantes.

19.1. ☒ ¿La clase **String** en Java es mutable o inmutable?

String en Java es una clase **inmutable**.

Esto significa que:

- Una vez creada una cadena, **su contenido no puede cambiar.**
- Cualquier "modificación" (como concatenar, sustituir, recortar...) **crea un nuevo objeto String**, no modifica el existente.

19.2. ☒ ¿Qué ocurre al concatenar dos cadenas?

Cuando haces:

```
String s = "Hola";  
s = s + " mundo";
```

En realidad pasa esto:

1. Se **crea un nuevo objeto** `String` con el contenido `"Hola mundo"`.
2. La variable `s` pasa a apuntar al nuevo objeto.
3. El objeto antiguo `"Hola"` queda sin usar (y lo recogerá el *garbage collector* más tarde).

Esto significa que **cada concatenación crea un objeto nuevo**, lo que puede ser **muy ineficiente** si haces muchas operaciones repetidas.

19.3. ¿Qué debemos hacer si vamos a concatenar muchas veces?

Usar `StringBuilder` (o `StringBuffer` si necesitas sincronización).

Ejemplo recomendado:

```
StringBuilder sb = new StringBuilder();

for (int i = 0; i < 10000; i++) {
    sb.append(i).append(", ");
}

String resultado = sb.toString();
```

Ventajas de `StringBuilder`:

- Es **mutable**.
- Permite **modificar la misma estructura** sin crear objetos nuevos.
- Para concatenaciones repetidas, es **mucho más eficiente** que usar `String`.

Resumen

Pregunta	Respuesta
¿ <code>String</code> es mutable o inmutable?	Immutable
¿Qué pasa al concatenar?	Se crea un nuevo <code>String</code> cada vez
¿Qué usar si concatenamos muchas veces?	<code>StringBuilder</code>

20. Comparación de objetos en POO y en Java

✓ 1. En POO, ¿se comparan objetos por contenido o por identidad?

Depende del lenguaje y del diseño de la clase **pero en general**:

- **Identidad** → si son **el mismo objeto** en memoria.
- **Contenido** → si **representan el mismo valor**.

En POO, por defecto, **dos objetos distintos son diferentes**, aunque tengan el mismo contenido. Para compararlos por contenido, debes **definir un criterio propio** (por ejemplo, implementando un método como `equals`).

✓ 2. ¿Qué es el método `equals` en Java?

`equals(Object o)` es un método heredado de la clase base `Object`.

Sirve para decidir si **dos objetos deben considerarse "iguales"** por su contenido o significado, no por su posición en memoria.

✓ 3. ¿Qué hace `equals` por defecto?

Por defecto, en `Object`, el método:

```
public boolean equals(Object obj) {  
    return (this == obj);  
}
```

Es decir:

👉 **Compara solo la identidad**, no el contenido.

Dos objetos diferentes siempre serán distintos, aunque tengan los mismos valores internos.

✓ 4. ¿Cómo se deben comparar dos cadenas (`String`) en Java?

`String` **sobrescribe** `equals` para que compare **contenido**, no identidad.

Por tanto, las cadenas se comparan así:

```
String a = "hola";  
String b = "hola";  
  
a.equals(b);    // ✓ TRUE → mismo contenido  
a == b;         // ✗ No usar (solo compara identidad)
```

En Java, las cadenas SIEMPRE deben compararse con `equals`, nunca con `==`.

★ Resumen final

Pregunta	Respuesta
¿Cómo se comparan objetos en POO?	Normalmente por identidad, salvo que la clase defina comparación por contenido.
¿Qué es <code>equals</code> ?	Método de <code>Object</code> usado para comparar objetos por contenido.
¿Qué hace por defecto?	Compara identidad (<code>this == obj</code>).
¿Cómo comparar dos <code>String</code> ?	Con <code>equals</code> , nunca con <code>==</code> .

21. ¿Qué son las clases "wrapper" en un lenguaje de programación orientado a objetos? ¿Cómo se hace? ¿Es un proceso automático? ¿Qué ventajas tienen? ¿Todos los lenguajes orientados a objetos tienen tipos primitivos y necesitan wrappers?

Respuesta

Aquí tienes una explicación clara, ordenada y breve sobre **clases wrapper**, enfocada en lo que suele pedirse en teoría de Java y POO:

21. ☒ ¿Qué son las clases *wrapper* en un lenguaje orientado a objetos?

Una **clase wrapper** (o "envoltorio") es una clase que **representa un tipo primitivo como un objeto**.

En Java, los tipos primitivos (como `int`, `double`, `boolean`...) **no son objetos**, así que necesitan una clase que los envuelva para poder tratarlos como objetos.

Ejemplos:

Tipo primitivo	Clase wrapper
<code>int</code>	<code>Integer</code>
<code>double</code>	<code>Double</code>
<code>boolean</code>	<code>Boolean</code>
<code>char</code>	<code>Character</code>

☒ ¿Cómo se hace?

En Java, hay dos formas:

◇ a) Manualmente (antes de Java 5)

```
int x = 5;
Integer obj = new Integer(x); // envoltura manual
```

◇ b) Automáticamente (desde Java 5): Autoboxing

```
Integer obj = 5; // Java convierte int → Integer automáticamente
int y = obj; // unboxing: Integer → int
```

☑ ¿Es un proceso automático?

Sí.

Desde Java 5 existe:

- **Autoboxing** → convertir de tipo primitivo a wrapper.
- **Unboxing** → convertir de wrapper a primitivo.

Ejemplo típico:

```
List<Integer> lista = new ArrayList<>();
lista.add(10); // autoboxing: int → Integer
int n = lista.get(0); // unboxing
```

☑ ¿Qué ventajas tienen las clases wrapper?

✓ 1. Permiten almacenar tipos primitivos en colecciones

Las colecciones (`ArrayList`, `Map`, etc.) solo aceptan **objetos**, no primitivas.

```
List<Integer> lista = new ArrayList<>();
```

✓ 2. Permiten tener valores nulos

Un `int` no puede ser `null`, pero un `Integer` sí.

✓ 3. Ofrecen métodos adicionales

Por ejemplo:

```
int x = Integer.parseInt("123");  
String s = Integer.toString(50);
```

✓ 4. Facilitan trabajar con APIs que requieren objetos

Como reflexión, genéricos, colecciones, etc.

☑ ¿Todos los lenguajes OO tienen tipos primitivos y necesitan wrappers?

No.

Depende del lenguaje:

◇ Lenguajes que SÍ tienen tipos primitivos y necesitan wrappers

- **Java** (int vs Integer, double vs Double)
- **C#** (int vs Int32, double vs Double)

◇ Lenguajes que NO tienen tipos primitivos: todo son objetos

- **Python**
- **Ruby**
- **Smalltalk**

En estos lenguajes **no existen wrappers**, porque **no existen tipos primitivos separados**.

Un 5 es un objeto tan válido como cualquier otro.

★ Resumen final

- Una clase *wrapper* es una clase que envuelve un tipo primitivo para tratarlo como objeto.
- En Java existen 8 wrappers para los 8 tipos primitivos.
- Java hace la conversión automáticamente (*autoboxing* y *unboxing*).
- Las wrappers permiten usar números en colecciones, tener valores nulos y usar métodos útiles.
- No todos los lenguajes necesitan wrappers: solo los que distinguen entre primitivos y objetos.

☑ 22.1. En POO, ¿qué es un *tipo de dato enumerado*?

Un **tipo enumerado** (o *enum*) es un tipo de dato que **solo puede tomar un conjunto finito y fijo de valores posibles**.

Ejemplos conceptuales:

- Los días de la semana (**LUNES**, **MARTES**, ...)
- Los colores básicos (**ROJO**, **VERDE**, **AZUL**)
- Las direcciones (**NORTE**, **SUR**, ...)

Sirven para **modelar conceptos que tienen un número limitado de valores correctos**, evitando errores como usar cadenas arbitrarias o números mágicos.

☑ 22.2. En Java, ¿un tipo enumerado es una clase?

Sí.

En Java, un **enum** es realmente una **clase** especial que:

- Extiende implícitamente `java.lang.Enum`
- Tiene instancias *predefinidas* (sus constantes)
- Es más potente que los **enum** en otros lenguajes como C

Ejemplo simple:

```
public enum Color {  
    ROJO, VERDE, AZUL;  
}
```

Aquí, `Color.ROJO`, `Color.VERDE` y `Color.AZUL` son **objetos únicos** de la clase `Color`.

☑ 22.3. ¿Qué ventajas tienen los enumerados en Java en términos de encapsulación?

Los **enum** en Java aportan varias ventajas importantes relacionadas con la **encapsulación**:

✓ 1. Seguridad del tipo (type safety)

Solo se pueden usar los valores definidos en el enumerado.

Evita errores como:

```
String dia = "Arbisdía"; // ✗ no tiene sentido
```

En su lugar:

```
DiaSemana dia = DiaSemana.LUNES; // ✓
```

✓ 2. Control total sobre las instancias

No se pueden crear nuevos valores fuera de los declarados:
la propia clase enum controla cuántas instancias existen.
Esto refuerza las **invariantes** del tipo.

✓ 3. Permiten añadir métodos, atributos y lógica interna

Los **enum** pueden encapsular comportamiento, igual que una clase normal.

```
public enum Direccion {  
    NORTE(0, 1),  
    SUR(0, -1),  
    ESTE(1, 0),  
    OESTE(-1, 0);  
  
    private final int dx;  
    private final int dy;  
  
    private Direccion(int dx, int dy) {  
        this.dx = dx;  
        this.dy = dy;  
    }  
  
    public int getDx() { return dx; }  
    public int getDy() { return dy; }  
}
```

Esto permite esconder detalles internos (**encapsulación**) y exponer solo lo necesario.

✓ 4. No se pueden modificar: son inmutables

Cada valor del **enum** es un objeto **inmutable**, lo cual facilita:

- mantener invariantes,
- evitar errores de estado,
- usar valores en colecciones sin riesgos.

✓ 5. Pueden tener constructores privados

Esto refuerza la encapsulación porque solo los valores declarados son válidos.

★ Resumen final

- En POO, un **enumerado** define un tipo con un conjunto fijo de valores.
- En **Java**, un **enum** es realmente una **clase** especial con instancias controladas.
- Los enumerados mejoran la **encapsulación** porque:
 - garantizan seguridad del tipo,

- limitan estrictamente las instancias,
- pueden ocultar detalles internos,
- son inmutables,
- y permiten asociar comportamiento sin exponer la implementación.

23. Crea un tipo enumerado en Java que se llame **Mes**, con doce posibles instancias y que además proporcione métodos para obtener cuántos días tiene ese mes, el ordinal de ese mes en el año (1-12), empleando atributos privados y constructores del tipo enumerado. Añade además cuatro métodos para devolver si ese mes tiene algunos días de invierno, primavera, verano u otoño, indicando con un booleano el hemisferio (norte o sur, parámetro **enHemisferioNorte**).

¡Aquí tienes un **enum Mes** en Java con las doce instancias, días por mes (incluyendo caso bisiesto), ordinal 1-12 y los métodos de estación por hemisferio usando **atributos privados** y **constructores** del propio **enum**:

```
public enum Mes {  
    ENERO(31),  
    FEBRERO(28, 29),  
    MARZO(31),  
    ABRIL(30),  
    MAYO(31),  
    JUNIO(30),  
    JULIO(31),  
    AGOSTO(31),  
    SEPTIEMBRE(30),  
    OCTUBRE(31),  
    NOVIEMBRE(30),  
    DICIEMBRE(31);  
  
    // Atributos privados  
    private final int diasNoBisiesto;  
    private final int diasBisiesto;  
  
    // Constructores privados del enum  
    private Mes(int dias) {  
        this.diasNoBisiesto = dias;  
        this.diasBisiesto = dias; // igual si no es febrero  
    }  
  
    private Mes(int diasNoBisiesto, int diasBisiesto) {  
        this.diasNoBisiesto = diasNoBisiesto;  
        this.diasBisiesto = diasBisiesto;  
    }  
  
    // Métodos públicos  
  
    /** Días del mes en un año no bisiesto. */  
    public int dias() {  
        return diasNoBisiesto;  
    }  
}
```

```
/** Días del mes según si el año es bisiesto. */
public int dias(boolean esAnioBisiesto) {
    return esAnioBisiesto ? diasBisiesto : diasNoBisiesto;
}

/** Ordinal 1..12 del mes en el año. */
public int numeroMes() {
    return this.ordinal() + 1;
}

public boolean esDePrimavera(boolean esHemisferioNorte) {
    return esHemisferioNorte
        ? (this == MARZO || this == ABRIL || this == MAYO)
        : (this == SEPTIEMBRE || this == OCTUBRE || this == NOVIEMBRE);
}

public boolean esDeVerano(boolean esHemisferioNorte) {
    return esHemisferioNorte
        ? (this == JUNIO || this == JULIO || this == AGOSTO)
        : (this == DICIEMBRE || this == ENERO || this == FEBRERO);
}

public boolean esDeOtoño(boolean esHemisferioNorte) {
    return esHemisferioNorte
        ? (this == SEPTIEMBRE || this == OCTUBRE || this == NOVIEMBRE)
        : (this == MARZO || this == ABRIL || this == MAYO);
}

public boolean esDeInvierno(boolean esHemisferioNorte) {
    return esHemisferioNorte
        ? (this == DICIEMBRE || this == ENERO || this == FEBRERO)
        : (this == JUNIO || this == JULIO || this == AGOSTO);
}
}
```